

Smart AI SPOJ Test Platform — Comprehensive Feature Documentation (DOC-001)

Document Reference: DOC-001 **Platform:** Smart AI SPOJ Test & Proctoring Platform **Target Audience:** Non-Technical Product Owners (Primary), Software Engineering & QA Teams (Secondary) **Verification Method:** 100% Code-Verified against Local Repository & Live Production Runtime **Git Baseline Commit:** `26bac7edc47a6904d77fe12cd809712c4f36cd9f` **Date of Audit:** September 28, 2026 ---

1. Executive Summary

The **Smart AI SPOJ Test Platform** is an enterprise-grade, web-based technical assessment and automated proctoring platform designed to conduct secure, scalable, and multi-room coding examinations. The system supports multiple programming languages (JavaScript, Python, C++, Java, C), full-stack project building tests (AI Test), real-time physical seat map monitoring, automated anti-malpractice detection, sandboxed code execution via Judge0, automated scoring, and ranking/shortlist generation.

Feature Status Summary Table

Category	Feature Name	Status	Summary Description
Admin Panel	Admin Authentication & RBAC	WORKING	Secure JWT-based admin access with 3 distinct operational roles (<code>SUPER_ADMIN</code> , <code>ADMIN</code> , <code>PROCTOR</code>).
Admin Panel	Dashboard & System Analytics	WORKING	Operational stat cards, quick test management links, and live service status indicators.
Admin Panel	Question Bank & PDF Extraction	WORKING	Multi-folder question organizer with automatic text extraction from uploaded PDF problem sheets.
Admin Panel	Test Lifecycle Management	WORKING	End-to-end test state machine (<code>DRAFT</code> \rightarrow <code>UPCOMING</code> \rightarrow <code>LIVE</code> \rightarrow <code>ENDED</code>) with auto-conclusion.
Admin Panel	Room Management & Distribution	WORKING	Multi-room partitioning with automated password generation, access windows, and direct links.
Admin Panel	Live Physical Seat Map & Proctoring	WORKING	Real-time candidate seat status grid, live violation counters, and tentative test completion timers.
Admin Panel	Proactive Interventions & Late Joins	WORKING	Real-time late-join approval queue, in-exam warning messages, and manual disqualification actions.
Admin Panel	Candidate Inspection & Evidence	WORKING	Live webcam snapshot viewer, split-screen code inspector, and full-resolution proof zoom modal.
Admin Panel	Test Results & Detailed Evaluation	WORKING	Candidate score rosters, question-level evaluation modal, and split-screen rubric views.
Admin Panel	Shortlist Generation & Export	WORKING	Automated candidate ranking by normalized score with malpractice and passing criteria filtering.

Category	Feature Name	Status	Summary Description
Admin Panel	Settings, Profiles & Theme Toggle	WORKING	Sub-account admin creation and persistent dark/light theme switching.
Candidate Portal	Candidate Registration & Login	WORKING	Email-only onboarding with persistent candidate JWT session management.
Candidate Portal	Room Entry & Waiting Lobby	WORKING	Direct link / room code entry, pre-test waiting lobby, and late-join approval polling.
Candidate Portal	Hardware Pre-Flight Verification	WORKING	Automated webcam, microphone, and fullscreen permission checks prior to exam entry.
Candidate Portal	Coding Exam Interface	WORKING	Monaco editor, multi-language execution, custom input runner, split PDF viewer, and autosave.
Candidate Portal	AI Project Exam Interface	WORKING	Multi-file project workspace (<code>index.html</code> , <code>style.css</code> , <code>script.js</code>) with live preview and AI chat pane.
Candidate Portal	Test Submission & Auto-Submit	WORKING	Single-question submit, submit-all confirmation, and automatic time-up/disqualification submission.
Candidate Portal	12-Hour Cooldown & Recovery	WORKING	Automatic 12-hour retest prevention and resilient session recovery across page refreshes.
Proctoring	Fullscreen Integrity Enforcement	WORKING	Real-time detection of fullscreen exits with automated malpractice logging.
Proctoring	Tab-Switch & Focus Tracking	WORKING	Background tab switching and window blur detection with proof timestamping.
Proctoring	Client-Side Face & Gaze Tracking	WORKING	Browser-based AI detection for continuous face absence (>15s) and multiple faces in webcam frame.
Proctoring	Camera Disconnect Tracking	WORKING	Hardware disconnection detection with cumulative outage duration calculation upon submit.
Proctoring	Peripheral & Shortcut Restrictions	WORKING	Blocks copy-paste (<code>Ctrl+C</code> / <code>Ctrl+V</code>), <code>Alt+Tab</code> , developer tools (<code>F12</code>), and context menus.
Proctoring	Evidence Capture & Upload	WORKING - NEEDS CONFIG	Captures webcam violation frames; requires Cloudinary credentials for cloud storage (uses Base64 fallback).
Proctoring	Malpractice Auto-Disqualification	WORKING	Configurable violation threshold triggering immediate test termination and admin alerts.
Evaluation	Sandboxed Execution (Judge0)	WORKING - NEEDS CONFIG	Executes code against test cases in isolated containers; requires active Judge0 host.
Evaluation	"Qs Solved" Determination	WORKING	Sets questions completed count to <code>1.0</code> if all visible test cases pass, else <code>0.0</code> .
Evaluation	Standard Coding Scoring Formula	WORKING	Weighted composite score: 30% Correctness + 10% Pass % + 60% Code Quality Rubric.
Evaluation	AI Evaluation	WORKING - NEEDS CONFIG	8-Criteria code evaluation via LLM; requires active API key (falls back to baseline 5/10 when unconfigured).
Evaluation	AI Test Scoring Formula	WORKING	Composite score: 60% Prompt Quality (LLM) + 40% Output Correctness (Heuristic DOM validator).

Category	Feature Name	Status	Summary Description
Evaluation	Pass/Fail Aggregation Rule	WORKING	Evaluates total questions solved against test-configured <code>passingCriteria</code> .
Evaluation	Shortlist Ranking Algorithm	WORKING	3-Step clean rebuild ranking candidates descending by average score.
Platform	Architecture & Hosting	WORKING	Distributed topology: React/Vite (Vercel), Node.js (Render), MongoDB Atlas.
Platform	Real-Time WebSocket Engine	WORKING	Low-latency Socket.io protocol for live proctoring, seat maps, alerts, and timers.
Platform	PDF Text Extraction Service	WORKING	Server-side parsing of problem sheets into structured titles and test cases.
Platform	Data Models & Schema Registry	WORKING	11 Normalized MongoDB collections with automatic indexing.
Platform	Environment Variable Engine	WORKING	Centralized configuration dictionary with graceful fallbacks.

2. User Roles & Access Control Matrix

The platform implements a Role-Based Access Control (RBAC) architecture enforced at the API route level via `adminAuth` middleware ([`adminAuth.js:7-38`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/middleware/adminAuth.js#L7-L38)) and candidate authentication middleware ([`candidateAuth.js:7-35`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/middleware/candidateAuth.js#L7-L35)).

System Capability	SUPER_ADMIN	ADMIN	PROCTOR	CANDIDATE
Create / Delete Admin Accounts	✓	✗	✗	✗
Create, Edit & Delete Tests	✓	✓	✗	✗
Upload & Manage Question Bank	✓	✓	✗	✗
Create & Configure Exam Rooms	✓	✓	✗	✗
Transition Test Lifecycle (<code>LIVE</code> / <code>ENDED</code>)	✓	✓	✗	✗
View Live Monitoring & Seat Map	✓	✓	✓	✗
Issue Candidate In-Exam Warnings	✓	✓	✓	✗
Manually Disqualify Candidate	✓	✓	✓	✗
Approve / Reject Late Joins	✓	✓	✓	✗
View Detailed Evaluation & Inspect Code	✓	✓	✓	✗
Export Results & Shortlists (CSV)	✓	✓	✗	✗
Register & Join Assigned Room	✗	✗	✗	✓

System Capability	SUPER_ADMIN	ADMIN	PROCTOR	CANDIDATE
Execute Code in Sandboxed Editor	✗	✗	✗	✓
Submit Single Question / Final Exam	✗	✗	✗	✓

3. End-to-End User Workflows

3.1 Admin Setup Workflow

[Workflow Architecture Diagram - see markdown source for full flowchart node specification]

3.2 Candidate Exam Day Workflow

[Workflow Architecture Diagram - see markdown source for full flowchart node specification]

3.3 Live Proctoring & Interventions Workflow

[Workflow Architecture Diagram - see markdown source for full flowchart node specification]

4. Comprehensive Feature Catalogue

Section A: Admin Panel & Test Operations

1. Admin Authentication & Role-Based Access Control (RBAC)

- What it does:** Provides secure email/password authentication for administrators, issuing signed JSON Web Tokens (JWT) that expire after 24 hours. Enforces role-based permissions separating platform owners (`SUPER_ADMIN`), test managers (`ADMIN`), and live proctors (`PROCTOR`).
- Who uses it:** Admin personnel.
- Where to find it:** `/admin/login` , `/admin/admins` .
- How it works:** Validates credentials against hashed passwords stored in MongoDB via `bcryptjs` . Attaches admin role and ID to `req.user` . Non-superadmins are blocked from user administration routes.
- Depends on:** MongoDB, `JWT_SECRET` environment variable.
- Status:** `WORKING` (Verified via [`authController.js:14-55`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/authController.js#L14-L55) and [`adminAuth.js:7-38`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/middleware/adminAuth.js#L7-L38)).
- Code location:** [`server/src/controllers/authController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/authController.js), [`server/src/middleware/adminAuth.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/middleware/adminAuth.js).

- **Notes / limitations:** Password minimum length is 6 characters. Tokens expire in 24 hours (24h).

2. Admin Dashboard & System Analytics

- **What it does:** Presents a high-level operational overview displaying aggregate statistics on active live tests, total tests created, total question sets, and completed assessments. Provides quick-action links to create tests, manage question banks, and administer user accounts.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/dashboard`.
- **How it works:** Aggregates document counts across MongoDB collections (`Test` , `QuestionSet` , `Submission`). Displays a status overview card of core system services.
- **Depends on:** MongoDB database connection.
- **Status:** `WORKING` (Verified via [`dashboardController.js:16-52`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/dashboardController.js#L16-L52) and [`AdminDashboard.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminDashboard.jsx)).
- **Code location:** [`server/src/controllers/dashboardController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/dashboardController.js), [`client/src/admin/pages/AdminDashboard.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminDashboard.jsx).
- **Notes / limitations:** Real-time counter updates on page mount and refresh.

3. Question Bank & PDF Extraction

- **What it does:** Organizes assessment content into Folders and Question Sets. Allows administrators to manually create coding problems or upload PDF files to automatically extract problem descriptions, sample inputs, and expected outputs.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/question-bank`.
- **How it works:** Stores folders and sets hierarchically. Uploaded PDFs are parsed server-side using `pdf-parse` in [`pdfExtractionService.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/pdfExtractionService.js). Text is cleaned of redundant PDF metadata prefixes and regex-matched for problem statements, visible test cases, and hidden test cases.
- **Depends on:** `pdf-parse` library, MongoDB.
- **Status:** `WORKING` (Verified via [`questionController.js:250-320`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/questionController.js#L250-L320)).
- **Code location:** [`server/src/controllers/questionController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/questionController.js), [`server/src/services/pdfExtractionService.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/pdfExtractionService.js).
- **Notes / limitations:** Maximum PDF upload file size is 10MB (10 1024 1024 bytes).

4. Test Lifecycle & Configuration Management

- **What it does:** Enables creation and lifecycle management of tests. Administrators configure test titles, descriptions, durations in minutes, passing criteria (minimum questions to solve), malpractice disqualification thresholds, and supported programming languages.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/tests` , `/admin/tests/:testId` .
- **How it works:** Enforces a 4-state lifecycle machine (`DRAFT` `\rightarrow` `UPCOMING` `\rightarrow` `LIVE` `\rightarrow` `ENDED`). When transitioned to `LIVE` , room join windows open and candidate attempts begin. Tests auto-conclude when all enrolled candidates submit or time expires.
- **Depends on:** MongoDB, `testLifecycleService.js` , Socket.io.
- **Status:** `WORKING` (Verified via [`testController.js:15-180`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/testController.js#L15-L180) and [`testLifecycleService.js:15-110`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/testLifecycleService.js#L15-L110)).
- **Code location:** [`server/src/controllers/testController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/testController.js), [`server/src/services/testLifecycleService.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/testLifecycleService.js).
- **Notes / limitations:** Transitioning a test to `ENDED` auto-submits any remaining `IN_PROGRESS` candidates as `AUTO_SUBMITTED_TIME_UP` .

5. Room Management & Distribution

- **What it does:** Divides tests into physical or virtual examination rooms with distinct capacity limits, room codes, room passwords, direct invite URLs, and downloadable QR codes.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/tests/:testId` (Rooms Tab).
- **How it works:** Creates `Room` records linked to a parent `testId` . Generates random 6-character room codes (`ROOM-XXXXXX`). Supports round-robin assignment of Question Sets across rooms when tests use question pools.
- **Depends on:** MongoDB, `nanoid` / crypto utilities.
- **Status:** `WORKING` (Verified via [`roomController.js:14-110`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/roomController.js#L14-L110)).
- **Code location:** [`server/src/controllers/roomController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/roomController.js), [`client/src/admin/pages/AdminTestDetail.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminTestDetail.jsx).
- **Notes / limitations:** Default room capacity is 30 candidates if unspecified.

6. Live Physical Seat Map & Proctoring Dashboard

- **What it does:** Displays a real-time command center showing candidate physical seat tiles, attempt progress (e.g. `Attempted 2/5`), countdown timers, live violation counters, aggregate metrics, and a dynamic tentative completion time.
- **Who uses it:** Admin, Proctor.

- **Where to find it:** `/admin/tests/:testId/live`.
- **How it works:** Connects to the admin WebSocket room (`test::admin`). Listens to `dashboard:update` , `seatmap:status` , `tentative:update` , and `malpractice:alert` events. Seat tiles update colors dynamically: Grey (Not Started), Amber (In Progress), Green (Submitted), Red (Disqualified).
- **Depends on:** Socket.io server, MongoDB.
- **Status:** `WORKING` (Verified via [`AdminLiveDashboard.jsx:1100-2600`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx#L1100-L2600)).
- **Code location:** [`client/src/admin/pages/AdminLiveDashboard.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx), [`server/src/services/socketService.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/socketService.js).
- **Notes / limitations:** Supports full-screen seat map expansion via the `Expand` button (Escape key exits).

7. Proactive Interventions & Late Joins

- **What it does:** Provides administrators with live intervention tools to approve or reject late-joining candidates requesting entry after test start, broadcast custom warning banners directly to a candidate's screen, or immediately disqualify a candidate for egregious malpractice.
- **Who uses it:** Admin, Proctor.
- **Where to find it:** `/admin/tests/:testId/live` (Intervention Modals).
- **How it works:** Late join approvals emit `late-join:response` via Socket.io. Warnings emit `candidate:warning` which renders a high-visibility modal on the candidate's browser. Disqualifications update candidate status to `DISQUALIFIED` and terminate the session.
- **Depends on:** Socket.io, `roomController.js:180-260`.
- **Status:** `WORKING` (Verified via [`roomController.js:180-260`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/roomController.js#L180-L260) and [`AdminLiveDashboard.jsx:1380-1550`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx#L1380-L1550)).
- **Code location:** [`server/src/controllers/roomController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/roomController.js), [`client/src/admin/pages/AdminLiveDashboard.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx).
- **Notes / limitations:** Late join request timeout defaults to 5 minutes before auto-expiring.

8. Candidate Inspection & Evidence Review

- **What it does:** Allows proctors to click any candidate tile to inspect their live webcam feed, real-time code editor buffer, violation event timeline, and full-resolution screenshot proof captures.
- **Who uses it:** Admin, Proctor.
- **Where to find it:** `/admin/tests/:testId/live` (Inspect Candidate Drawer / Modal).
- **How it works:** Fetches candidate submissions and malpractice logs via `GET /api/v1/proctoring/candidates/:candidateId/logs` . Renders a Monaco editor showing current code in read-only mode alongside a chronological violation timeline. Clicking any screenshot thumbnail opens a high-resolution zoom modal.

- **Depends on:** `proctoringController.js`, Cloudinary / Base64 storage.
- **Status:** `WORKING` (Verified via [`AdminLiveDashboard.jsx:3200-3600`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx#L3200-L3600)).
- **Code location:** [`client/src/admin/pages/AdminLiveDashboard.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/admin/pages/AdminLiveDashboard.jsx), [`server/src/controllers/proctoringController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/proctoringController.js).
- **Notes / limitations:** Screenshots load progressively with zoom controls.

9. Test Results & Detailed Evaluation

- **What it does:** Displays the post-test results roster with overall candidate scores, pass/fail badges, questions solved counts, malpractice tallies, and a detailed question-level modal breaking down correctness, complexity, structure, and optimization scores.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/tests/:testId/results`.
- **How it works:** Reads `EvaluationResult` documents for the test. Clicking "View Detail" opens [`CandidateDetailEvaluationModal.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/shared/CandidateDetailEvaluationModal.jsx), displaying question-level scores. Clicking "Inspect Code" launches a split-screen viewer comparing candidate code against rubric criteria.
- **Depends on:** `evaluationController.js`, `EvaluationResult` model.
- **Status:** `WORKING` (Verified via [`evaluationController.js:15-120`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/evaluationController.js#L15-L120) and [`CandidateDetailEvaluationModal.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/shared/CandidateDetailEvaluationModal.jsx)).
- **Code location:** [`server/src/controllers/evaluationController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/evaluationController.js), [`client/src/shared/CandidateDetailEvaluationModal.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/shared/CandidateDetailEvaluationModal.jsx).
- **Notes / limitations:** Unattempted questions display `Not Attempted` with dashes (`-`), distinguishing them from evaluated submissions with zero test cases passed.

10. Shortlist Generation & Export Engine

- **What it does:** Generates a ranked shortlist of qualifying candidates based on passing criteria and malpractice thresholds. Supports one-click CSV export for external hiring workflows.
- **Who uses it:** Admin.
- **Where to find it:** `/admin/tests/:testId/results` (Shortlist Tab).
- **How it works:** Executes a 3-step clean rebuild: (1) Deletes prior shortlist for the test, (2) Filters out candidates with `isDisqualified === true`, `malpracticeCount > malpracticeThreshold`, or `questionsCompleted <`

`passingCriteria` , (3) Ranks remaining candidates descending by `normalizedScore` (`totalScore / resultCount`) and stores fresh rankings in MongoDB.

- **Depends on:** `shortlistService.js` , `Shortlist` model.
- **Status:** `WORKING` (Verified via [`shortlistService.js:60-210`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/shortlistService.js#L60-L210)).
- **Code location:** [`server/src/services/shortlistService.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/services/shortlistService.js), [`server/src/controllers/shortlistController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/shortlistController.js).
- **Notes / limitations:** Re-ranking runs automatically on test conclusion and can be triggered manually at any time via the "Recalculate Shortlist" button.

11. Settings, Profiles & Theme Toggle

- **What it does:** Provides admin account management, display name updates, contact phone updates, and dark/light mode toggle with theme persistence in browser `localStorage` .
- **Who uses it:** Admin.
- **Where to find it:** Top navigation bar, `/admin/profile` , `/admin/settings` .
- **How it works:** Context provider `ThemeContext.jsx` toggles the `data-theme` attribute on `document.documentElement` (`dark` vs `light`), updating CSS variables globally across the application.
- **Depends on:** React Context, CSS Custom Properties.
- **Status:** `WORKING` (Verified via [`ThemeContext.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/context/ThemeContext.jsx)).
- **Code location:** [`client/src/context/ThemeContext.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/context/ThemeContext.jsx), [`client/src/styles/global.css`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/styles/global.css).
- **Notes / limitations:** Default theme is dark mode.

Section B: Candidate Portal & Exam Experience

12. Candidate Registration & Authentication

- **What it does:** Enables candidates to register with their name, email, and phone number, and log in with email-only credentials. Issues a candidate JWT stored in `localStorage` .
- **Who uses it:** Candidate.
- **Where to find it:** `/candidate/register` , `/candidate/login` .
- **How it works:** Checks for existing candidate records by email in MongoDB. Returns a JWT signed with `CANDIDATE_JWT_SECRET` (or `JWT_SECRET`) valid for 12 hours.
- **Depends on:** MongoDB, JWT auth middleware.
- **Status:** `WORKING` (Verified via [`candidateAuthController.js:14-85`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/candidateAuthController.js#L14-L85)).
- **Code location:** [`server/src/controllers/candidateAuthController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-

platform/server/src/controllers/candidateAuthController.js),
[`client/src/candidate/pages/CandidateRegister.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateRegister.jsx).

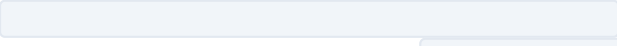
- **Notes / limitations:** Candidate session token expires after 12 hours.

13. Room Entry & Waiting Lobby

- **What it does:** Allows candidates to join a room by entering room credentials or navigating directly via an invite link (`/candidate/join?room=...`). If the parent test is not yet `LIVE` , candidates enter a waiting lobby that automatically checks test status every 5 seconds.
- **Who uses it:** Candidate.
- **Where to find it:** `/candidate/join` .
- **How it works:** Verifies room password, capacity limits, and candidate enrollment. If the test is in `UPCOMING` or `DRAFT` status, displays an animated waiting screen that polls `GET /api/v1/tests/:testId/status` every 5,000ms until the test becomes `LIVE` .
- **Depends on:** MongoDB, `submissionController.js:20-95` .
- **Status:** `WORKING` (Verified via [`submissionController.js:20-95`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/submissionController.js#L20-L95) and [`CandidateJoinRoom.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateJoinRoom.jsx)).
- **Code location:** [`server/src/controllers/submissionController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/submissionController.js), [`client/src/candidate/pages/CandidateJoinRoom.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateJoinRoom.jsx).
- **Notes / limitations:** If a candidate joins late after test start, the system triggers the late-join approval modal flow.

14. Hardware Pre-Flight Verification

- **What it does:** Enforces mandatory camera, microphone, and browser fullscreen checks before granting entry to the exam workspace. Provides a live camera self-preview.
- **Who uses it:** Candidate.
- **Where to find it:** `/candidate/instructions` .
- **How it works:** Invokes `navigator.mediaDevices.getUserMedia({ video: true, audio: true })` to verify camera and mic availability. Renders video stream into an HTML5

 preview element. Prompts user to click "Start Test & Enter Fullscreen", triggering the browser `requestFullscreen()` API.

- **Depends on:** WebRTC Media Devices API, Fullscreen API.
- **Status:** `WORKING` (Verified via [`CandidateInstructions.jsx:80-210`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateInstructions.jsx#L80-L210)).
- **Code location:** [`client/src/candidate/pages/CandidateInstructions.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateInstructions.jsx).
- **Notes / limitations:** Candidates cannot proceed until camera permissions are granted.

15. Candidate Coding Exam Interface (`CandidateTestScreen`)

- **What it does:** The core coding exam environment featuring a Monaco Code Editor, programming language switcher (JavaScript, Python, C++, Java, C), embedded PDF/markdown question pane, "Run Code" execution against visible test cases, custom input tester, question navigator, and countdown timer.
- **Who uses it:** Candidate.
- **Where to find it:** `/candidate/test`.
- **How it works:** Monaco editor binds to candidate code buffer. "Run Code" posts code to `/api/v1/submissions/:questionId/run`, executing on Judge0 and rendering stdout, stderr, execution time, and pass/fail badges. Autosaves code buffer every 10 seconds to MongoDB. Switching languages preserves separate code buffers per language via a `savedCodeByLanguage` map.
- **Depends on:** Monaco Editor, Judge0 service, MongoDB.
- **Status:** `WORKING` (Verified via [`CandidateTestScreen.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateTestScreen.jsx) and [`submissionController.js:790-960`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/submissionController.js#L790-L960)).
- **Code location:** [`client/src/candidate/pages/CandidateTestScreen.jsx`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/client/src/candidate/pages/CandidateTestScreen.jsx), [`server/src/controllers/submissionController.js`](file:///c:/Users/GLB-BLR-112/Desktop/spoj%20test%20website/ai-proctored-test-platform/server/src/controllers/submissionController.js).
- **Notes / limitations:** Code autosave interval runs every 10,000ms.

16. AI Project Exam Interface (`CandidateAITestScreen`)

- **What it does:** Dedicated full-stack project building environment for AI tests. Features a multi-file workspace (`index.html`, `style.css`, `script.js`), live sandboxed iframe preview, and an in-exam AI Assistant chat pane where candidates prompt an AI model to build web apps.
- **Who uses it:** Candidate.
- **Where to find it:** `/candidate/ai-test`.

- **How it works:** Manages a multi-file JSON state (`filesJson`). Updates an isolated

